AI-Native Engineering Transformation

Detailed implementation plan | Confluence PRD to validated GitHub pull request

Purpose: Establish a repeatable, secure, human-governed delivery pipeline spanning requirements, Jira planning, repository-aware implementation, testing, and review. This document is a proposed plan, not a description of an already deployed system.

Initial scope: Airflow, backend services, and frontend micro-frontends (MFE). Target: Review-ready PRs, not automatic production deployment.

1. Executive decision and scope

- Adopt a staged 16-week pilot with explicit exit criteria. Start with one low-risk repository and a narrow class of changes; expand only after measured success.
- Separate external-safe PRDs (what and why) from restricted technical plans (how, where, credentials, internal architecture).
- Use approval gates before implementation of high-risk changes and before merging. Keep a complete audit trail of agent actions.
- Treat AI-generated code as untrusted until tested, scanned, reviewed, and approved.

2. End-to-end workflow

Approved Confluence PRD → Requirements validation → Jira epic/stories → Repository impact analysis → Technical plan and approval → Isolated branches → AI implementation → Automated validation → GitHub PR(s) with evidence → Human review → Knowledge capture.

Workflow state machine

DRAFT_PRD → REQUIREMENTS_REVIEW → READY_FOR_PLANNING → PLAN_APPROVAL → IMPLEMENTING → VALIDATING → PR_OPEN → HUMAN_REVIEW → DONE. Exception states: BLOCKED, NEEDS_CLARIFICATION, FAILED_VALIDATION, CANCELLED. Every transition stores actor, timestamp, source links, and outcome.

3. Recommended system architecture

Layer	Services / duties
Source systems	Confluence pages, Jira issues, GitHub repositories, CI checks and approved architecture documents
Orchestration	Durable workflow engine, event handlers, idempotency keys, state storage, retries, approvals and audit log
AI agents	Requirements analyst, solution planner, repository investigator, coding agent, QA agent, PR reviewer
Execution	Ephemeral isolated workers with restricted tokens, network allowlists and per-repository permissions
Evidence / memory	Artifact store for plans and logs; indexed approved patterns, CI fixes and historical PR decisions

For the pilot, a modular application plus a durable job queue is adequate. Avoid deploying many microservices until throughput, security boundaries, or operational needs justify them.

4. Six-phase delivery roadmap

Phase	Timing	Deliverables / exit criteria
1. Engineering standards	Weeks 1-2	Baseline lint, tests, secret scanning, branch protection, reusable CI workflows; passing reference pipeline
2. PRD to Jira	Weeks 3-4	PRD template, completeness validator, Jira issue creation with duplicate prevention; human-approved test run
3. Repository intelligence	Weeks 5-7	Repo manifests, architecture map, dependency discovery, read-only plan agent; engineering sign-off on plans
4. Coding to PR	Weeks 8-11	Sandboxed implementation worker, branch creation, change limits, PR creation and evidence bundle
5. Quality and learning	Weeks 12-14	CI repair loop, quality dashboard, review feedback ingestion, curated reusable solution library
6. Pilot and scale	Weeks 15-16	Controlled live pilot, baseline comparison, security review, go/no-go decision and expansion backlog

5. Phase 1: standardize repositories and quality gates

Actions

- Inventory repositories: languages, build commands, test frameworks, branch rules, CODEOWNERS, CI status, dependencies and deployment owners.

- Define language-specific baseline: Python (Ruff or Pylint/Flake8 according to team policy), unit tests and coverage; JavaScript/TypeScript (ESLint, type checking, unit tests); Airflow DAG parsing and import tests.
- Create reusable GitHub Actions workflows for lint, tests, dependency scanning, secret scanning and SonarQube; pin third-party actions and use least-privilege workflow permissions.
- Enable required status checks and human approvals; establish an exceptions register for legacy repos with existing violations.

Exit criteria

At least one representative repository per workstream has reproducible local and CI checks; required checks are enforced; a baseline failure inventory and ownership map exist.

6. Phase 2: Confluence PRD to Jira

PRD schema

Required sections: title, summary, context, problem, goals, non-goals, users, functional and non-functional requirements, user flow, Given/When/Then acceptance criteria, privacy/security, dependencies, open questions, rollout and success metrics.

Automation steps

- Trigger only on an explicit status such as "Ready for Engineering"; read page version and permission classification.
- Validate completeness and identify contradictions, missing roles, vague acceptance criteria, unresolved dependencies and risky data flows.
- Generate a proposed epic, stories, acceptance criteria, dependencies and validation plan; preserve links to exact PRD version and requirement IDs.
- Require product/engineering approval for ambiguous requirements; then create or update Jira items using an idempotency key based on page ID and approved version.
- Store traceability: PRD requirement ID → Jira story → implementation PR → test evidence.

Example Jira story

Summary: Show a safe access-unavailable state on the payroll workflow page.
Acceptance criteria: Given an ineligible role, when the page opens, then the user sees approved guidance without confidential details.
Technical notes: Restricted to engineering Jira fields; inspect existing permission behavior, do not change authorization policy.
Validation: Unit tests, role-based integration tests and browser verification.

7. Phase 3: repository-aware AI planning

- Maintain a repository manifest describing ownership, language, entry points, test commands, API contracts, dependency boundaries and prohibited paths.
- Provide read-only access to source, architecture decision records, coding standards and approved examples. Search relevant files before proposing changes.

- Produce a structured plan: affected repos/files, expected behavior, contract changes, test plan, migration needs, rollout and rollback, risk level, unanswered questions.
- For multi-repository features, identify sequencing and compatibility requirements; prefer backward-compatible backend changes before frontend adoption.
- Block implementation if a requirement is ambiguous or requires an unapproved architecture/security decision.

Plan review rubric

Reviewers verify traceability to acceptance criteria, minimal change scope, compatibility, security, test coverage, operational impact and whether a simpler approach exists.

8. Phase 4: controlled code generation and PR creation

Worker lifecycle

- Provision an ephemeral sandbox; check out an approved base commit; create a ticket-linked feature branch.
- Give the agent a bounded task with permitted repositories, file paths, tools, time budget and maximum change size.
- Implement one coherent story at a time; require tests alongside changes and preserve existing public contracts unless explicitly approved.
- Run local checks; permit a limited number of repair attempts; stop and escalate on repeated failures or unexplained changes.
- Push through a short-lived GitHub App token; open a draft PR with ticket links, change summary, test evidence, risk notes and reviewer requests.

Multi-repo coordination

Use one Jira epic with repo-specific child stories and separate PRs. Track dependency order, required contract tests and compatible release sequencing. Avoid a single agent holding unrestricted write access to every repository.

9. Phase 5: validation, evidence and compound engineering

Validation matrix

Area	Required evidence
Airflow	DAG import/parse tests, scheduling and task dependency checks, representative task tests
Backend	Unit and API tests, schema compatibility, authorization checks, dependency/security scans
Frontend MFE	Type checks, unit/component tests, accessibility checks, browser workflow screenshots or recordings using synthetic data
Cross-repo	Contract tests, integration checks, version compatibility, feature flag and rollback readiness

PR evidence bundle

Include PRD/Jira references, approved plan version, commit SHA, changed components, acceptance-criteria-to-test mapping, CI run links, lint/scan summaries, known limitations, screenshots with test data, rollout notes and rollback steps.

Learning loop

- Collect merged PR patterns, failed CI diagnoses, reviewer corrections and incidents; remove secrets and confidential data before indexing.
- Store only verified, versioned solutions with repository scope, applicable versions, provenance, owner and review date.
- Retrieve relevant examples during planning; do not treat historical code or comments as authoritative instructions.

10. Security, privacy and governance

- Use least-privilege GitHub App, Jira and Confluence credentials; separate read and write permissions; rotate short-lived credentials.
- Treat PRDs, code comments, issue text and retrieved documents as untrusted input; defend against prompt injection and prohibit secret exfiltration.
- Run agents in network-restricted, ephemeral environments; prevent access to production secrets and customer data.
- Require approvals for database migrations, authorization changes, infrastructure changes, sensitive data processing and high-impact cross-repo work.
- Log prompts/outputs and tool actions only under an approved retention and redaction policy; define incident response and emergency shutdown.
- Never automatically merge or deploy in the pilot. Branch protections and required human review remain authoritative.

11. Operating model and responsibilities

Role	Accountability
Product owner	PRD completeness, user outcomes, acceptance criteria, final wording
Engineering lead	Technical plan approval, architecture constraints, code ownership
Platform/DevEx	Orchestrator, integrations, sandbox, reusable CI, observability
Security	Threat model, credential boundaries, access review and policy
QA	Test strategy, evidence validity, regression coverage
Developers	Review generated PRs, resolve complex changes, approve merge

12. Metrics and pilot evaluation

Metric	Measurement
PRD-to-PR lead time	Median hours from approved PRD to review-ready PR, versus manual baseline
First-pass CI success	Share of AI PRs passing required checks without repair
Acceptance-criteria coverage	Share of approved criteria mapped to executable checks or documented manual validation
Human review burden	Median review minutes and number of requested change cycles
Rework / escaped defects	Post-merge defects and reopened stories per delivered change
Automation completion	Share of eligible stories producing a valid PR without human implementation intervention

Illustrative pilot thresholds (agree with stakeholders before launch): at least 90% PRD-to-story traceability; 100% required security gates executed; zero unreviewed merges; measurable reduction in median lead time without higher defect rates. Do not optimize speed at the expense of quality.

13. Pilot design and go/no-go criteria

- Select 10-20 low-to-medium risk stories with clear acceptance criteria and representative Airflow/backend/frontend tasks; exclude major migrations and critical authorization changes initially.
- Capture a manual baseline using comparable recent tickets, including lead time, CI failures, review effort and defects.
- Run AI-generated work under normal branch protection and reviewer policies; compare against baseline by task complexity.
- Expand only when quality, security, traceability and review burden meet agreed thresholds. Record failure cases as prioritized platform improvements.

14. First 10 working days: actionable checklist

Day	Action / output
1-2	Name executive sponsor, product owner, platform lead and pilot repositories; agree success criteria
3-4	Audit CI, branch protections, permissions, repository documentation and data classification
5	Publish external-safe PRD template and internal technical-plan template
6-7	Create repository manifests, reusable check templates and sample acceptance-criteria mapping
8	Implement Confluence approval trigger and Jira draft-generation proof of concept
9	Threat-model the agent workflow and define human approval checkpoints
10	Demo one approved PRD converted to Jira and a read-only implementation plan; review phase-2 backlog

15. Risks and mitigations

Risk	Mitigation
Ambiguous PRDs	Completeness scoring, clarification gate, approved PRD versions
Incorrect code changes	Small scoped tasks, read-only planning, isolated workers, mandatory review
CI flakiness	Baseline flaky tests, classify infra versus code failures, bounded retries
Cross-repo breaking changes	Contract tests, explicit dependency graph and release sequencing
Secrets / data exposure	Least privilege, sandbox egress controls, synthetic fixtures, redacted evidence
Low developer trust	Transparent plans, reproducible test evidence, metrics and opt-in pilot

16. Definition of done for a generated PR

- Every change maps to approved Jira acceptance criteria and an immutable PRD version.
- All required checks pass, or documented exceptions have explicit human approval.
- Tests cover positive, negative, error and authorization cases where relevant.
- PR description includes evidence, risks, rollout/rollback instructions and links to source work.
- Required code owners approve; the system records the final outcome and reusable lessons.

Recommended next decision: Approve the pilot repositories, identify the integration owner for Confluence/Jira/GitHub, and select the first three example PRDs. Begin with read-only planning before granting any write access.